

CounterExample Guided Abstraction Refinement (CEGAR)

Guest Lecture for Formal Verification

Sankalp Gambhir

November 25, 2025

EPFL

Checking Safety with Abstract Interpretation

Simple branching program. Concrete state sets $\subseteq \mathbb{Z}^3$, abstract states $\in \mathcal{I}(\mathbb{Z})^3$.

E.g. $\alpha(\{(0, 0, 0), (1, 2, 3)\}) = ([0, 1], [0, 2], [0, 3])$

Checking Safety with Abstract Interpretation

Simple branching program. Concrete state sets $\subseteq \mathbb{Z}^3$, abstract states $\in \mathcal{I}(\mathbb{Z})^3$.

E.g. $\alpha(\{(0, 0, 0), (1, 2, 3)\}) = ([0, 1], [0, 2], [0, 3])$

```
1  x := havoc();
2  y := 0; z := 0;
3
4  if (x >= 0) then
5      y := 1;
6  else
7      z := 1;
8
9
10 assert(y + z <= 1);
```

Checking Safety with Abstract Interpretation

Simple branching program. Concrete state sets $\subseteq \mathbb{Z}^3$, abstract states $\in \mathcal{I}(\mathbb{Z})^3$.

E.g. $\alpha(\{(0, 0, 0), (1, 2, 3)\}) = ([0, 1], [0, 2], [0, 3])$

```
1  x := havoc();
2  y := 0; z := 0;    // (-inf, inf), [0, 0], [0, 0]
3
4  if (x >= 0) then
5      y := 1;
6  else
7      z := 1;
8
9
10 assert(y + z <= 1);
```

Checking Safety with Abstract Interpretation

Simple branching program. Concrete state sets $\subseteq \mathbb{Z}^3$, abstract states $\in \mathcal{I}(\mathbb{Z})^3$.

E.g. $\alpha(\{(0, 0, 0), (1, 2, 3)\}) = ([0, 1], [0, 2], [0, 3])$

```
1  x := havoc();
2  y := 0; z := 0;    // (-inf, inf), [0, 0], [0, 0]
3
4  if (x >= 0) then  // [0, inf), [0, 0], [0, 0]
5      y := 1;
6  else
7      z := 1;
8
9
10 assert(y + z <= 1);
```

Checking Safety with Abstract Interpretation

Simple branching program. Concrete state sets $\subseteq \mathbb{Z}^3$, abstract states $\in \mathcal{I}(\mathbb{Z})^3$.

E.g. $\alpha(\{(0, 0, 0), (1, 2, 3)\}) = ([0, 1], [0, 2], [0, 3])$

```
1  x := havoc();
2  y := 0; z := 0;    // (-inf, inf), [0, 0], [0, 0]
3
4  if (x >= 0) then    // [0, inf), [0, 0], [0, 0]
5      y := 1;         // [0, inf), [1, 1], [0, 0]
6  else
7      z := 1;
8
9
10 assert(y + z <= 1);
```

Checking Safety with Abstract Interpretation

Simple branching program. Concrete state sets $\subseteq \mathbb{Z}^3$, abstract states $\in \mathcal{I}(\mathbb{Z})^3$.

E.g. $\alpha(\{(0, 0, 0), (1, 2, 3)\}) = ([0, 1], [0, 2], [0, 3])$

```
1  x := havoc();
2  y := 0; z := 0;    // (-inf, inf), [0, 0], [0, 0]
3
4  if (x >= 0) then    // [0, inf), [0, 0], [0, 0]
5      y := 1;         // [0, inf), [1, 1], [0, 0]
6  else                // (-inf, -1], [0, 0], [0, 0]
7      z := 1;
8
9
10 assert(y + z <= 1);
```

Checking Safety with Abstract Interpretation

Simple branching program. Concrete state sets $\subseteq \mathbb{Z}^3$, abstract states $\in \mathcal{I}(\mathbb{Z})^3$.

E.g. $\alpha(\{(0, 0, 0), (1, 2, 3)\}) = ([0, 1], [0, 2], [0, 3])$

```
1  x := havoc();
2  y := 0; z := 0;    // (-inf, inf), [0, 0], [0, 0]
3
4  if (x >= 0) then    // [0, inf), [0, 0], [0, 0]
5      y := 1;         // [0, inf), [1, 1], [0, 0]
6  else                // (-inf, -1], [0, 0], [0, 0]
7      z := 1;         // (-inf, -1], [0, 0], [1, 1]
8
9
10 assert(y + z <= 1);
```

Checking Safety with Abstract Interpretation

Simple branching program. Concrete state sets $\subseteq \mathbb{Z}^3$, abstract states $\in \mathcal{I}(\mathbb{Z})^3$.

E.g. $\alpha(\{(0, 0, 0), (1, 2, 3)\}) = ([0, 1], [0, 2], [0, 3])$

```
1  x := havoc();
2  y := 0; z := 0;    // (-inf, inf), [0, 0], [0, 0]
3
4  if (x >= 0) then    // [0, inf), [0, 0], [0, 0]
5      y := 1;          // [0, inf), [1, 1], [0, 0]
6  else                // (-inf, -1], [0, 0], [0, 0]
7      z := 1;          // (-inf, -1], [0, 0], [1, 1]
8
9  // (-inf, inf), [0, 1], [0, 1]
10 assert(y + z <= 1);
```

Relational Analysis

```
1  x := havoc();  
2  y := 0; z := 0;  
3  
4  if (x >= 0) then  
5      y := 1;  
6  else  
7      z := 1;  
8  
9  
10 assert(y + z <= 1);
```

Relational Analysis

```
1  x := havoc();  
2  y := 0; z := 0; // P0: y0 == 0 && z0 == 0  
3  
4  if (x >= 0) then  
5      y := 1;  
6  else  
7      z := 1;  
8  
9  
10 assert(y + z <= 1);
```

Relational Analysis

```
1  x := havoc();
2  y := 0; z := 0; // P0:  $y_0 == 0 \ \&\& \ z_0 == 0$ 
3
4  if (x >= 0) then
5      y := 1;          // P1:  $x_0 == 0 \implies y_1 == 1 \ \&\& \ (x_1, z_1) == (x_0, z_0)$ 
6  else
7      z := 1;
8
9
10 assert(y + z <= 1);
```

Relational Analysis

```
1  x := havoc();
2  y := 0; z := 0; // P0: y0 == 0 && z0 == 0
3
4  if (x >= 0) then
5      y := 1;      // P1: x0 == 0 ==> y1 == 1 && (x1, z1) == (x0, z0)
6  else
7      z := 1;      // P2: x0 != 0 ==> z1 == 1 && (x1, y1) == (x0, y0)
8
9
10 assert(y + z <= 1);
```

Relational Analysis

```
1  x := havoc();
2  y := 0; z := 0; // P0: y0 == 0 && z0 == 0
3
4  if (x >= 0) then
5      y := 1;      // P1: x0 == 0 ==> y1 == 1 && (x1, z1) == (x0, z0)
6  else
7      z := 1;      // P2: x0 != 0 ==> z1 == 1 && (x1, y1) == (x0, y0)
8
9  // P0 && P1 && P2
10 assert(y + z <= 1);
```

Check if $P0 \wedge P1 \wedge P2 \implies y_1 + z_1 \leq 1$.

Relational Analysis

```
1  x := havoc();
2  y := 0; z := 0; // P0: y0 == 0 && z0 == 0
3
4  if (x >= 0) then
5      y := 1;      // P1: x0 == 0 ==> y1 == 1 && (x1, z1) == (x0, z0)
6  else
7      z := 1;      // P2: x0 != 0 ==> z1 == 1 && (x1, y1) == (x0, y0)
8
9  // P0 && P1 && P2
10 assert(y + z <= 1);
```

Check if $P0 \wedge P1 \wedge P2 \implies y_1 + z_1 \leq 1$.

Easy to do with an SMT solver!

Relational Analysis

```
1  x := havoc();
2  y := 0; z := 0; // P0: y0 == 0 && z0 == 0
3
4  if (x >= 0) then
5      y := 1;      // P1: x0 == 0 ==> y1 == 1 && (x1, z1) == (x0, z0)
6  else
7      z := 1;      // P2: x0 != 0 ==> z1 == 1 && (x1, y1) == (x0, y0)
8
9  // P0 && P1 && P2
10 assert(y + z <= 1);
```

Check if $P0 \wedge P1 \wedge P2 \implies y_1 + z_1 \leq 1$.

Easy to do with an SMT solver! Here, we get the formula is valid, so the program is safe.

Relational Analysis

```
1  x := havoc();
2  y := 0; z := 0; // P0: y0 == 0 && z0 == 0
3
4  if (x >= 0) then
5      y := 1;      // P1: x0 == 0 ==> y1 == 1 && (x1, z1) == (x0, z0)
6  else
7      z := 1;      // P2: x0 != 0 ==> z1 == 1 && (x1, y1) == (x0, y0)
8
9  // P0 && P1 && P2
10 assert(y + z <= 1);
```

Check if $P0 \wedge P1 \wedge P2 \implies y_1 + z_1 \leq 1$.

Easy to do with an SMT solver! Here, we get the formula is valid, so the program is safe.

What if we asserted $y + z == 2$ instead?

Relational Analysis

```
1  x := havoc();
2  y := 0; z := 0;
3
4  while (x > 0) do
5      x := x - 1;
6      if (x > 0) then
7          y := 1;
8      else
9          z := 1;
10
11
12  assert(y + z <= 1);
```

Relational Analysis

```
1  x := havoc();  
2  y := 0; z := 0;  $\Delta_{y=0, z=0}$   
3  
4  while (x > 0) do  
5      x := x - 1;  
6      if (x > 0) then  
7          y := 1;  
8      else  
9          z := 1;  
10  
11  
12  assert(y + z <= 1);
```

Relational Analysis

```
1  x := havoc();  
2  y := 0; z := 0;  $\Delta_{y=0, z=0}$   
3  
4  while (x > 0) do  $\Delta_{x>0}$   
5      x := x - 1;  
6      if (x > 0) then  
7          y := 1;  
8      else  
9          z := 1;  
10  
11  
12  assert(y + z <= 1);
```

Relational Analysis

1	x := havoc();	
2	y := 0; z := 0;	$\Delta_{y=0, z=0}$
3		
4	while (x > 0) do	$\Delta_{x>0}$
5	x := x - 1;	$\{((x, y, z), (x - 1, y, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
6	if (x > 0) then	
7	y := 1;	
8	else	
9	z := 1;	
10		
11		
12	assert(y + z <= 1);	

Relational Analysis

1	<code>x := havoc();</code>	
2	<code>y := 0; z := 0;</code>	$\Delta_{y=0, z=0}$
3		
4	<code>while (x > 0) do</code>	$\Delta_{x>0}$
5	<code>x := x - 1;</code>	$\{((x, y, z), (x - 1, y, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
6	<code>if (x > 0) then</code>	$\Delta_{x>0}$
7	<code>y := 1;</code>	
8	<code>else</code>	
9	<code>z := 1;</code>	
10		
11		
12	<code>assert(y + z <= 1);</code>	

Relational Analysis

1	<code>x := havoc();</code>	
2	<code>y := 0; z := 0;</code>	$\Delta_{y=0,z=0}$
3		
4	<code>while (x > 0) do</code>	$\Delta_{x>0}$
5	<code>x := x - 1;</code>	$\{((x, y, z), (x - 1, y, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
6	<code>if (x > 0) then</code>	$\Delta_{x>0}$
7	<code>y := 1;</code>	$\{((x, y, z), (x, 1, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
8	<code>else</code>	
9	<code>z := 1;</code>	
10		
11		
12	<code>assert(y + z <= 1);</code>	

Relational Analysis

1	<code>x := havoc();</code>	
2	<code>y := 0; z := 0;</code>	$\Delta_{y=0,z=0}$
3		
4	<code>while (x > 0) do</code>	$\Delta_{x>0}$
5	<code>x := x - 1;</code>	$\{((x, y, z), (x - 1, y, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
6	<code>if (x > 0) then</code>	$\Delta_{x>0}$
7	<code>y := 1;</code>	$\{((x, y, z), (x, 1, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
8	<code>else</code>	
9	<code>z := 1;</code>	
10		
11		
12	<code>assert(y + z <= 1);</code>	

Relational Analysis

1	<code>x := havoc();</code>	
2	<code>y := 0; z := 0;</code>	$\Delta_{y=0,z=0}$
3		
4	<code>while (x > 0) do</code>	$\Delta_{x>0}$
5	<code>x := x - 1;</code>	$\{((x, y, z), (x - 1, y, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
6	<code>if (x > 0) then</code>	$\Delta_{x>0}$
7	<code>y := 1;</code>	$\{((x, y, z), (x, 1, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
8	<code>else</code>	
9	<code>z := 1;</code>	$\Delta_{x \leq 0} \circ \{((x, y, z), (x, y, 1)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
10		
11		
12	<code>assert(y + z <= 1);</code>	

Relational Analysis

1	<code>x := havoc();</code>	
2	<code>y := 0; z := 0;</code>	$\Delta_{y=0, z=0}$
3		
4	<code>while (x > 0) do</code>	$\Delta_{x>0}$
5	<code>x := x - 1;</code>	$\{((x, y, z), (x - 1, y, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
6	<code>if (x > 0) then</code>	$\Delta_{x>0}$
7	<code>y := 1;</code>	$\{((x, y, z), (x, 1, z)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
8	<code>else</code>	
9	<code>z := 1;</code>	$\Delta_{x \leq 0} \circ \{((x, y, z), (x, y, 1)) \in \mathbb{Z}^3 \times \mathbb{Z}^3\}$
10		
11		$\Delta_{x \leq 0}$
12	<code>assert(y + z <= 1);</code>	

Relational Analysis

```
1  x := havoc();  
2  y := 0; z := 0;  $\Delta_0$   
3  
4  while (x > 0) do  $r_{\text{body}}$   
5      x := x - 1;  
6      if (x > 0) then  
7          y := 1;  
8      else  
9          z := 1;  
10  
11  $\Delta_1$   
12 assert(y + z <= 1);
```

Relational Analysis

```
1  x := havoc();  
2  y := 0; z := 0;  $\Delta_0$   
3  
4  while (x > 0) do  $r_{\text{body}}$   
5      x := x - 1;  
6      if (x > 0) then  
7          y := 1;  
8      else  
9          z := 1;  
10  
11  $\Delta_1$   
12 assert(y + z <= 1);
```

$$R = \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1$$

$$R = \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1$$

$$\begin{aligned} R &= \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1 \\ &= \Delta_0 \circ \left(\bigcup_{i=0}^{\infty} r_{\text{body}}^i \right) \circ \Delta_1 \end{aligned}$$

$$\begin{aligned} R &= \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1 \\ &= \Delta_0 \circ \left(\bigcup_{i=0}^{\infty} r_{\text{body}}^i \right) \circ \Delta_1 \\ &= \bigcup_{i=0}^{\infty} \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \end{aligned}$$

$$\begin{aligned} R &= \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1 \\ &= \Delta_0 \circ \left(\bigcup_{i=0}^{\infty} r_{\text{body}}^i \right) \circ \Delta_1 \\ &= \bigcup_{i=0}^{\infty} \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \end{aligned}$$

$$\begin{aligned} R &= \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1 \\ &= \Delta_0 \circ \left(\bigcup_{i=0}^{\infty} r_{\text{body}}^i \right) \circ \Delta_1 \\ &= \bigcup_{i=0}^{\infty} \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \end{aligned}$$

Do there exist $((x, y, z), (x', y', z')) = (s, s') \in R$ such that $y' + z' > 1$?

$$\begin{aligned} R &= \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1 \\ &= \Delta_0 \circ \left(\bigcup_{i=0}^{\infty} r_{\text{body}}^i \right) \circ \Delta_1 \\ &= \bigcup_{i=0}^{\infty} \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \end{aligned}$$

Do there exist $((x, y, z), (x', y', z')) = (s, s') \in R$ such that $y' + z' > 1$?

$$(s, s') \in R \wedge y' + z' > 1$$

$$\begin{aligned} R &= \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1 \\ &= \Delta_0 \circ \left(\bigcup_{i=0}^{\infty} r_{\text{body}}^i \right) \circ \Delta_1 \\ &= \bigcup_{i=0}^{\infty} \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \end{aligned}$$

Do there exist $((x, y, z), (x', y', z')) = (s, s') \in R$ such that $y' + z' > 1$?

$$(s, s') \in R \wedge y' + z' > 1$$

$$\iff (s, s') \in \left(\bigcup_{i=0}^{\infty} \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \right) \wedge y' + z' > 1$$

$$\begin{aligned} R &= \Delta_0 \circ r_{\text{body}}^* \circ \Delta_1 \\ &= \Delta_0 \circ \left(\bigcup_{i=0}^{\infty} r_{\text{body}}^i \right) \circ \Delta_1 \\ &= \bigcup_{i=0}^{\infty} \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \end{aligned}$$

Do there exist $((x, y, z), (x', y', z')) = (s, s') \in R$ such that $y' + z' > 1$?

$$(s, s') \in R \wedge y' + z' > 1$$

$$\iff (s, s') \in \left(\bigcup_{i=0}^{\infty} \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \right) \wedge y' + z' > 1$$

$$\iff \exists i \in \mathbb{N}. (s, s') \in \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right) \wedge y' + z' > 1$$

Theorem

If there exists a counterexample to the safety of the program, then there exists a finite execution demonstrating it.

Theorem

If there exists a counterexample to the safety of the program, then there exists a finite execution demonstrating it.

$$\left(\Delta_0 \circ r_{\text{body}}^0 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

Theorem

If there exists a counterexample to the safety of the program, then there exists a finite execution demonstrating it.

$$\left(\Delta_0 \circ r_{\text{body}}^0 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^1 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

Theorem

If there exists a counterexample to the safety of the program, then there exists a finite execution demonstrating it.

$$\left(\Delta_0 \circ r_{\text{body}}^0 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^1 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^2 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{SAT, error}$$

Theorem

If there exists a counterexample to the safety of the program, then there exists a finite execution demonstrating it.

$$\left(\Delta_0 \circ r_{\text{body}}^0 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^1 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^2 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{SAT, error}$$

Theorem

If there exists a counterexample to the safety of the program, then there exists a finite execution demonstrating it.

$$\left(\Delta_0 \circ r_{\text{body}}^0 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^1 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^2 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{SAT, error}$$

Note that this *definitively* implies that the program is unsafe.

Theorem

If there exists a counterexample to the safety of the program, then there exists a finite execution demonstrating it.

$$\left(\Delta_0 \circ r_{\text{body}}^0 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^1 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{UNSAT}$$

$$\left(\Delta_0 \circ r_{\text{body}}^2 \circ \Delta_1 \right) (s, s') \wedge y' + z' > 1 \implies \text{SAT, error}$$

Note that this *definitively* implies that the program is unsafe. But what if all of these were UNSAT?

“Bounded Model Checking”

- We are looking for a *model*, a concrete execution that violates safety.

“Bounded Model Checking”

- We are looking for a *model*, a concrete execution that violates safety.
- If there is a bug, there is a finite path demonstrating it.

“*Bounded Model Checking*”

- We are looking for a *model*, a concrete execution that violates safety.
- If there is a bug, there is a finite path demonstrating it.
- We are not listing *every* path, but incrementally exploring classes of paths.

“*Bounded Model Checking*”

- We are looking for a *model*, a concrete execution that violates safety.
- If there is a bug, there is a finite path demonstrating it.
- We are not listing *every* path, but incrementally exploring classes of paths.
- Bounded can mean different things in different contexts — loop unrollings, recursion depth, context switches, etc.

Exact Set of Reachable States

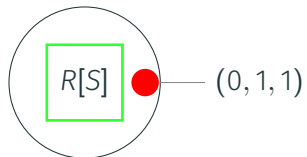


Safe States $\{(x, y, z) \mid y + z \leq 1\}$

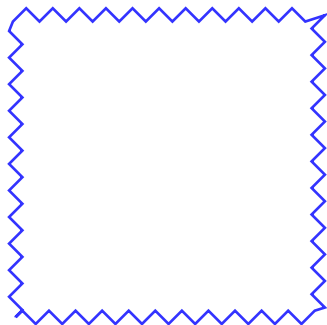
Exact Set of Reachable States



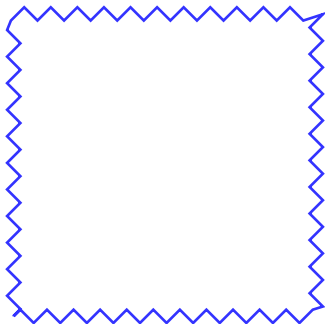
$R[S]$



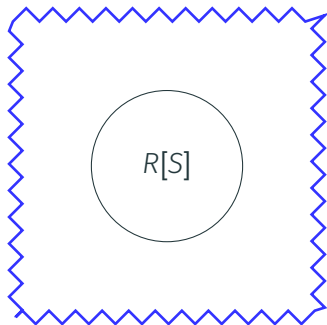
AI Overapproximation $\gamma(\alpha(R[S]))$



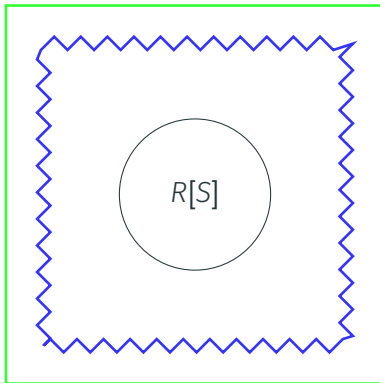
AI Overapproximation $\gamma(\alpha(R[S]))$
Invariant, concisely expressible



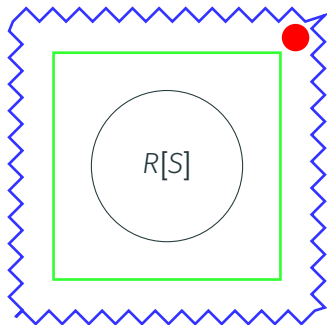
AI Overapproximation $\gamma(\alpha(R[S]))$
Invariant, concisely expressible



AI Overapproximation $\gamma(\alpha(R[S]))$
Invariant, concisely expressible



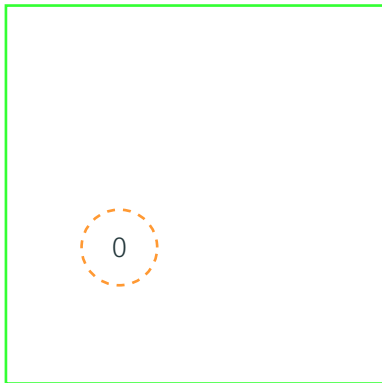
AI Overapproximation $\gamma(\alpha(R[S]))$
Invariant, concisely expressible



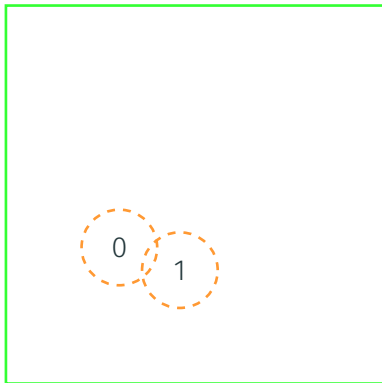
BMC Underapproximation $\bigcup_{i=0}^k \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right)$



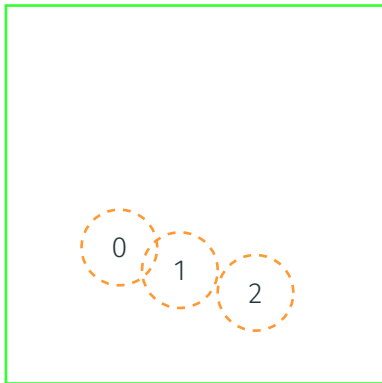
BMC Underapproximation $\bigcup_{i=0}^k \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right)$



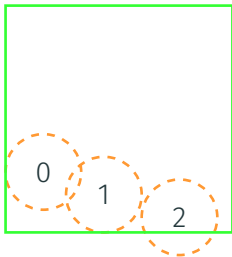
BMC Underapproximation $\bigcup_{i=0}^k \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right)$



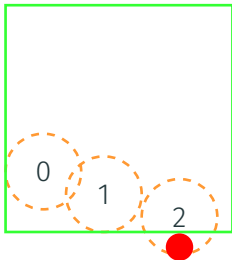
BMC Underapproximation $\bigcup_{i=0}^k \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right)$



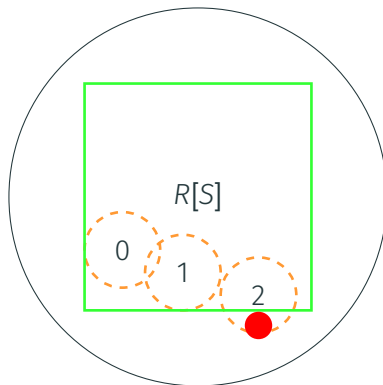
BMC Underapproximation $\bigcup_{i=0}^k (\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1)$



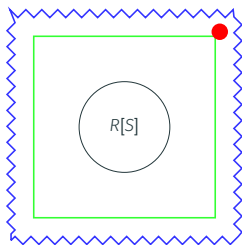
BMC Underapproximation $\bigcup_{i=0}^k \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right)$



BMC Underapproximation $\bigcup_{i=0}^k \left(\Delta_0 \circ r_{\text{body}}^i \circ \Delta_1 \right)$



Abstract Interpretation



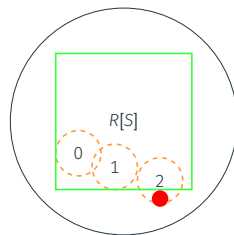
Computes Overapproximation

Success = safety

Failure = inconclusive

Proof of safety

Bounded Model Checking



Computes Underapproximation(s)

Success = inconclusive

Failure = bug found

Concrete counterexample trace

Investigating the Failure of Abstract Interpretation

```
1  x := havoc();  
2  y := 0; z := 0;  
3  
4  if (x >= 0) then  
5      y := 1;  
6  else  
7      z := 1;  
8  
9  
10 assert(y + z == 1);
```

Investigating the Failure of Abstract Interpretation

```
1  x := havoc();  
2  y := 0; z := 0;  
3  
4  if (x >= 0) then  
5      y := 1;  
6  else  
7      z := 1;  
8  
9  
10 assert(y + z == 1);
```

I can give you a great abstract domain that will prove this program safe:

Investigating the Failure of Abstract Interpretation

```
1  x := havoc();
2  y := 0; z := 0;
3
4  if (x >= 0) then
5      y := 1;
6  else
7      z := 1;
8
9
10 assert(y + z == 1);
```

I can give you a great abstract domain that will prove this program safe:

$$A = C = 2^{Z^3}, \alpha = \gamma = \text{id}_C.$$

Investigating the Failure of Abstract Interpretation

```
1  x := havoc();
2  y := 0; z := 0;
3
4  if (x >= 0) then
5      y := 1;
6  else
7      z := 1;
8
9
10 assert(y + z == 1);
```

I can give you a great abstract domain that will prove this program safe:

$$A = C = 2^{Z^3}, \alpha = \gamma = \text{id}_C.$$

The *method* is not insufficient, we are trading precision for performance.

Investigating the Failure of Abstract Interpretation

New domain: $\{n \leq x \leq m, n \leq y \leq m, n \leq z \leq m, n \leq y + z \leq m\}$.

```
1  x := havoc();
2  y := 0; z := 0;
3
4  if (x >= 0) then
5      y := 1;
6  else
7      z := 1;
8
9
10 assert(y + z == 1);
```

Investigating the Failure of Abstract Interpretation

New domain: $\{n \leq x \leq m, n \leq y \leq m, n \leq z \leq m, n \leq y + z \leq m\}$.

```
1  x := havoc();
2  y := 0; z := 0;    // 0 <= y + z <= 0
3
4  if (x >= 0) then
5      y := 1;
6  else
7      z := 1;
8
9
10 assert(y + z == 1);
```

Investigating the Failure of Abstract Interpretation

New domain: $\{n \leq x \leq m, n \leq y \leq m, n \leq z \leq m, n \leq y + z \leq m\}$.

```
1  x := havoc();
2  y := 0; z := 0;    // 0 <= y + z <= 0
3
4  if (x >= 0) then
5      y := 1;
6  else
7      z := 1;
8
9
10 assert(y + z == 1);
```

Investigating the Failure of Abstract Interpretation

New domain: $\{n \leq x \leq m, n \leq y \leq m, n \leq z \leq m, n \leq y + z \leq m\}$.

```
1  x := havoc();
2  y := 0; z := 0;    // 0 <= y + z <= 0
3
4  if (x >= 0) then
5      y := 1;        // 1 <= y + z <= 1
6  else
7      z := 1;
8
9
10 assert(y + z == 1);
```

Investigating the Failure of Abstract Interpretation

New domain: $\{n \leq x \leq m, n \leq y \leq m, n \leq z \leq m, n \leq y + z \leq m\}$.

```
1  x := havoc();
2  y := 0; z := 0;    // 0 <= y + z <= 0
3
4  if (x >= 0) then
5      y := 1;        // 1 <= y + z <= 1
6  else
7      z := 1;
8
9
10 assert(y + z == 1);
```

Investigating the Failure of Abstract Interpretation

New domain: $\{n \leq x \leq m, n \leq y \leq m, n \leq z \leq m, n \leq y + z \leq m\}$.

```
1  x := havoc();
2  y := 0; z := 0;    // 0 <= y + z <= 0
3
4  if (x >= 0) then
5      y := 1;        // 1 <= y + z <= 1
6  else
7      z := 1;        // 1 <= y + z <= 1
8
9
10 assert(y + z == 1);
```

Investigating the Failure of Abstract Interpretation

New domain: $\{n \leq x \leq m, n \leq y \leq m, n \leq z \leq m, n \leq y + z \leq m\}$.

```
1  x := havoc();
2  y := 0; z := 0;    // 0 <= y + z <= 0
3
4  if (x >= 0) then
5      y := 1;        // 1 <= y + z <= 1
6  else
7      z := 1;        // 1 <= y + z <= 1
8
9  // 1 <= y + z <= 1
10 assert(y + z == 1);
```

Investigating the Failure of Abstract Interpretation

How do we find the information our abstraction is missing?

Investigating the Failure of Abstract Interpretation

How do we find the information our abstraction is missing?

- Start with a better abstraction ✨

Investigating the Failure of Abstract Interpretation

How do we find the information our abstraction is missing?

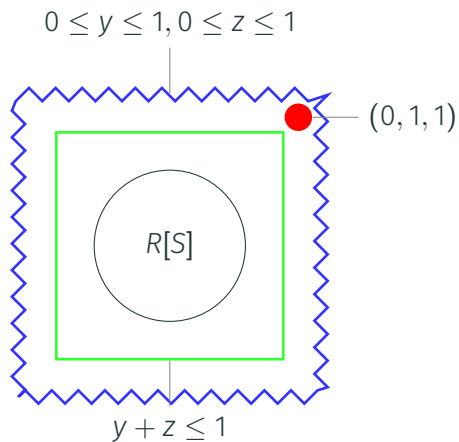
- Start with a better abstraction ✨
- Values and properties appearing in the program can be tracked

Investigating the Failure of Abstract Interpretation

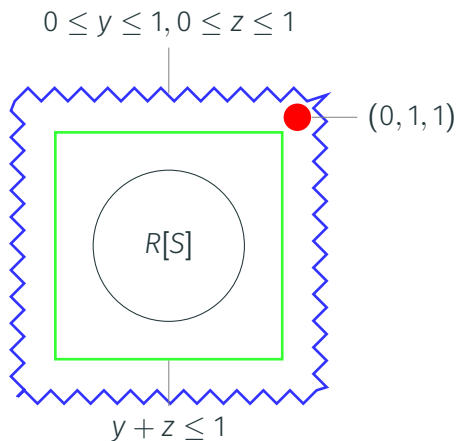
How do we find the information our abstraction is missing?

- Start with a better abstraction ✨
- Values and properties appearing in the program can be tracked
- Or, build a better abstraction 🛠️

What's Wrong with the Abstraction?

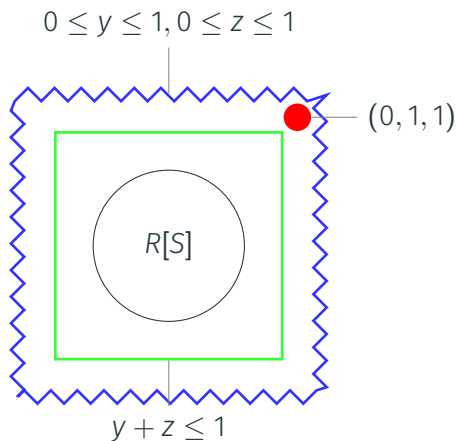


What's Wrong with the Abstraction?



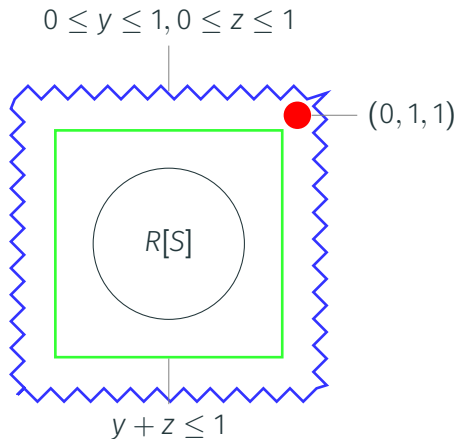
- Abstraction is too coarse. Needs to shrink!

What's Wrong with the Abstraction?



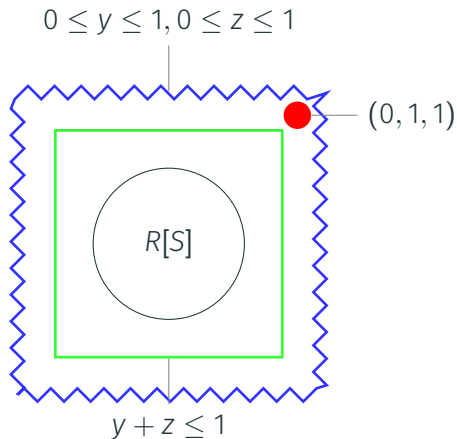
- Abstraction is too coarse. Needs to shrink!
- One obvious way: remove the point.

What's Wrong with the Abstraction?



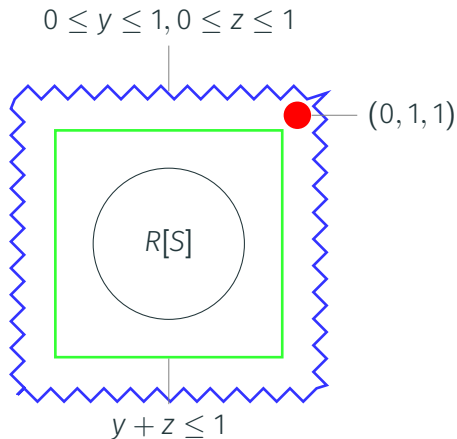
- Abstraction is too coarse. Needs to shrink!
- One obvious way: remove the point.
- End up with $0 \leq y \leq 1 \wedge 0 \leq z \leq 1 \wedge \neg(y = 1 \wedge z = 1)$

What's Wrong with the Abstraction?



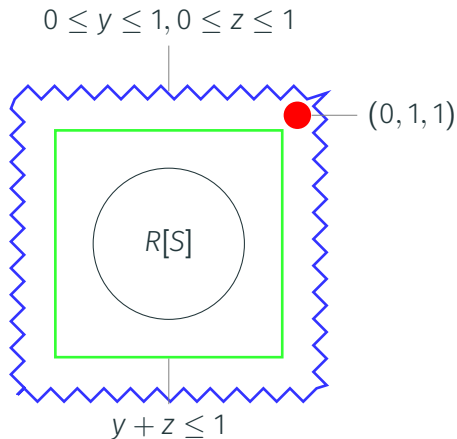
- Abstraction is too coarse. Needs to shrink!
- One obvious way: remove the point.
- End up with $0 \leq y \leq 1 \wedge 0 \leq z \leq 1 \wedge \neg(y = 1 \wedge z = 1)$
- One obvious problem too: possibly large number of such points.

What's Wrong with the Abstraction?



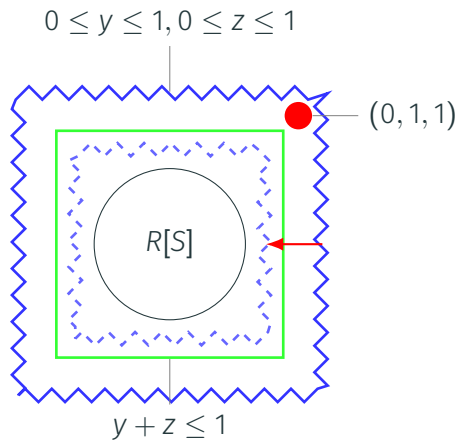
- Abstraction is too coarse. Needs to shrink!
- One obvious way: remove the point.
- End up with $0 \leq y \leq 1 \wedge 0 \leq z \leq 1 \wedge \neg(y = 1 \wedge z = 1)$
- One obvious problem too: possibly large number of such points.
- We used abstractions to avoid dealing with large sets in the first place!

What's Wrong with the Abstraction?

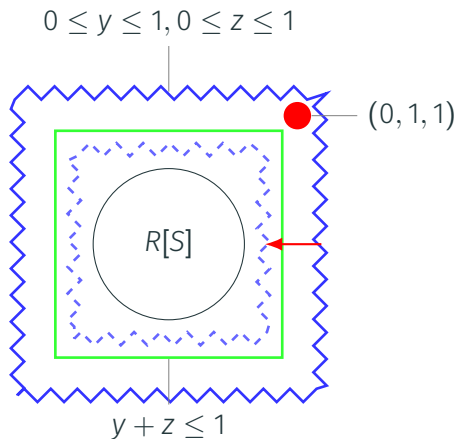


- Abstraction is too coarse. Needs to shrink!
- One obvious way: remove the point.
- End up with $0 \leq y \leq 1 \wedge 0 \leq z \leq 1 \wedge \neg(y = 1 \wedge z = 1)$
- One obvious problem too: possibly large number of such points.
- We used abstractions to avoid dealing with large sets in the first place!
- Abstract again 🔄

Abstraction Refinement (A Christmas Wishlist)

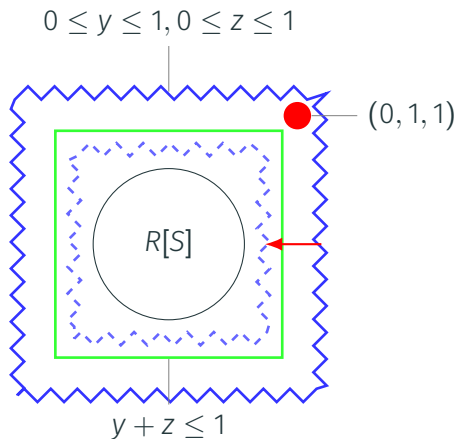


Abstraction Refinement (A Christmas Wishlist)



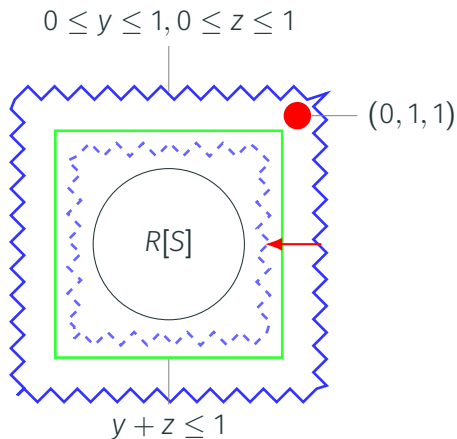
1. Generalize this reason to get a **set** of similar counterexamples.

Abstraction Refinement (A Christmas Wishlist)



1. Generalize this reason to get a **set** of similar counterexamples.
2. Learn **why** the counterexample fails the safety check.

Abstraction Refinement (A Christmas Wishlist)



1. Generalize this reason to get a **set** of similar counterexamples.
2. Learn **why** the counterexample fails the safety check.
3. **Refine** the abstraction to exclude this entire set.

Generalizing Counterexamples

Now, we take a step back to AI. We have, in principle, two programs, the *concrete* and the *abstract*. Counterexample from abstract program could be real or spurious. We need to check!

Now, we take a step back to AI. We have, in principle, two programs, the *concrete* and the *abstract*. Counterexample from abstract program could be real or spurious. We need to check!

Our two programs are two transition systems over states C and A , written with transfer functions T and $T^\#$ respectively.

Now, we take a step back to AI. We have, in principle, two programs, the *concrete* and the *abstract*. Counterexample from abstract program could be real or spurious. We need to check!

Our two programs are two transition systems over states C and A , written with transfer functions T and $T^\#$ respectively.

We know that there is an assertion failure in the abstract program. So, we will search for a program trace that actually produces this value.

Generalizing Counterexamples

(🔮) We will run BMC on the abstract program to find a program trace that can produce an assertion failure.

Generalizing Counterexamples

(🔮) We will run BMC on the abstract program to find a program trace that can produce an assertion failure.

This trace is a *finite, straight-line* program — no loops, no branches. It gives us a path through the control-flow graph.

Generalizing Counterexamples

(🔊) We will run BMC on the abstract program to find a program trace that can produce an assertion failure.

This trace is a *finite, straight-line* program — no loops, no branches. It gives us a path through the control-flow graph.

Example:

```
1  x := havoc();  
2  y := 0; z := 0;  
3  
4  assume(x >= 0); // then branch  
5  y := 1;  
6  
7  assert(y + z <= 1);
```

Generalizing Counterexamples

(🔮) We will run BMC on the abstract program to find a program trace that can produce an assertion failure.

This trace is a *finite, straight-line* program — no loops, no branches. It gives us a path through the control-flow graph.

Example:

```
1  x := havoc();  
2  y := 0; z := 0;  
3  
4  assume(x >= 0); // then branch  
5  y := 1;  
6  
7  assert(y + z <= 1);
```

Relational analysis (BMC with bound ∞) is complete for such programs! If there is a bug, we will find it. If there isn't, we get a proof of safety.

Generalizing Counterexamples

In this case, we get a proof of:

$$(x \geq 0 \wedge y = 1 \wedge z = 0) \implies y + z \leq 1$$

In general, this represents a *class* of spurious counterexamples.

In this case, we get a proof of:

$$(x \geq 0 \wedge y = 1 \wedge z = 0) \implies y + z \leq 1$$

In general, this represents a *class* of spurious counterexamples.

Crucially, we have found a gap between the abstract and concrete programs. We can remove these spurious counterexamples from the abstraction to make to more precise.

Generalizing Counterexamples

In this case, we get a proof of:

$$(x \geq 0 \wedge y = 1 \wedge z = 0) \implies y + z \leq 1$$

In general, this represents a *class* of spurious counterexamples.

Crucially, we have found a gap between the abstract and concrete programs. We can remove these spurious counterexamples from the abstraction to make to more precise.

Next, we need to figure out *why* this proof holds. Easier said than done!

Definition (Implicational Interpolant)

Given two formulas $A_{\bar{x},\bar{y}}$ and $B_{\bar{y},\bar{z}}$ such that $A_{\bar{x},\bar{y}} \implies B_{\bar{y},\bar{z}}$, an *interpolant* of the pair is a formula $I_{\bar{y}}$ such that:

$$A_{\bar{x},\bar{y}} \implies I_{\bar{y}}$$

$$I_{\bar{y}} \implies B_{\bar{y},\bar{z}}$$

Definition (Implicational Interpolant)

Given two formulas $A_{\bar{x},\bar{y}}$ and $B_{\bar{y},\bar{z}}$ such that $A_{\bar{x},\bar{y}} \implies B_{\bar{y},\bar{z}}$, an *interpolant* of the pair is a formula $I_{\bar{y}}$ such that:

$$A_{\bar{x},\bar{y}} \implies I_{\bar{y}}$$

$$I_{\bar{y}} \implies B_{\bar{y},\bar{z}}$$

An *interpolation procedure* for a class of formulas F is an algorithm that, given any $A, B \in F$ such that $A \implies B$, computes an interpolant I of the pair such that $I \in F$.

Definition (Implicational Interpolant)

Given two formulas $A_{\bar{x},\bar{y}}$ and $B_{\bar{y},\bar{z}}$ such that $A_{\bar{x},\bar{y}} \implies B_{\bar{y},\bar{z}}$, an *interpolant* of the pair is a formula $I_{\bar{y}}$ such that:

$$A_{\bar{x},\bar{y}} \implies I_{\bar{y}}$$

$$I_{\bar{y}} \implies B_{\bar{y},\bar{z}}$$

An *interpolation procedure* for a class of formulas F is an algorithm that, given any $A, B \in F$ such that $A \implies B$, computes an interpolant I of the pair such that $I \in F$.

Typically, F is taken to be the set of quantifier-free formulas in some first-order theory.

- Hard problem in general

Finding Interpolants

- Hard problem in general
- Usually: the interpolation procedure processes the proof of $A \implies B$ to construct an I

Finding Interpolants

- Hard problem in general
- Usually: the interpolation procedure processes the proof of $A \implies B$ to construct an I

Finding Interpolants

- Hard problem in general
- Usually: the interpolation procedure processes the proof of $A \implies B$ to construct an I

For now, use a bigger hammer to solve the problem. Claim:

- Given A and B , their interpolants form a lattice under implication.
- The strongest interpolant is $\exists \bar{x}. A_{\bar{x}, \bar{y}}$.
- The weakest interpolant is $\forall \bar{z}. B_{\bar{y}, \bar{z}}$.

Finding Interpolants

- Hard problem in general
- Usually: the interpolation procedure processes the proof of $A \implies B$ to construct an I

For now, use a bigger hammer to solve the problem. Claim:

- Given A and B , their interpolants form a lattice under implication.
- The strongest interpolant is $\exists \bar{x}. A_{\bar{x}, \bar{y}}$.
- The weakest interpolant is $\forall \bar{z}. B_{\bar{y}, \bar{z}}$.

Quantifiers are out of our expected class F .

Finding Interpolants

- Hard problem in general
- Usually: the interpolation procedure processes the proof of $A \implies B$ to construct an I

For now, use a bigger hammer to solve the problem. Claim:

- Given A and B , their interpolants form a lattice under implication.
- The strongest interpolant is $\exists \bar{x}. A_{\bar{x}, \bar{y}}$.
- The weakest interpolant is $\forall \bar{z}. B_{\bar{y}, \bar{z}}$.

Quantifiers are out of our expected class F . Use quantifier elimination ✨.

Proof of Interpolation

Claim: $\exists \bar{x}. A_{\bar{x}, \bar{y}}$ is an interpolant of $A_{\bar{x}, \bar{y}}$ and $B_{\bar{y}, \bar{z}}$. Further, for any interpolant $I_{\bar{y}}$, $I_{\bar{y}} \implies \exists \bar{x}. A_{\bar{x}, \bar{y}}$.

Interpolation:

$$\frac{}{A \vdash A} \text{Hyp} \quad \frac{}{A \vdash \exists \bar{x}. A} \exists R \quad \frac{\vdash A \implies B}{A \vdash B} \text{Restate} \quad \frac{A \vdash B}{\exists \bar{x}. A \vdash B} \exists L$$

Strongest interpolant:

$$\frac{A \vdash I}{\exists \bar{x}. A \vdash I} \exists L$$

(free variables to be checked to justify quantification steps)

- Propositional Logic: Samuel R Buss. **“Propositional proof complexity an introduction”**. In: *Computational Logic: Proceedings of the NATO Advanced Study Institute on Computational Logic, held in Marktoberdorf, Germany, July 29–August 10, 1997*. Springer, 1999, pp. 127–178
- Ortholattices: Simon Guilloud, Sankalp Gambhir, and Viktor Kunčák. **“Interpolation and Quantifiers in Ortholattices”**. In: *International Conference on Verification, Model Checking, and Abstract Interpretation*. Springer. 2023, pp. 235–257
- Scala implementation for ortholattices:
<https://github.com/sankalpgambhir/ol-interpolation>

Interpolation for our Counterexamples

We had:

$$(x \geq 0 \wedge y = 1 \wedge z = 0) \implies y + z \leq 1$$

Interpolants:

- $\exists x. A: y = 1 \wedge z = 0$
- $\forall. B: y + z \leq 1$

Note that the one below is what we wanted to make our analysis precise!

Interpolation for our Counterexamples

We had:

$$(x \geq 0 \wedge y = 1 \wedge z = 0) \implies y + z \leq 1$$

Interpolants:

- $\exists x. A: y = 1 \wedge z = 0$
- $\forall. B: y + z \leq 1$

Note that the one below is what we wanted to make our analysis precise! There isn't always an obvious choice of what the best interpolant is. The interpolant you end up with can depend on the theorem prover and interpolation procedure used.

Our interpolant represents the *inverse* of a set of spurious counterexamples. We can use it to refine our abstraction:

$$a \mapsto a \sqcap I$$

So in the last step, we get the refined abstraction:

$$0 \leq y \leq 1 \wedge 0 \leq z \leq 1 \wedge y + z \leq 1$$

which is sufficient to prove safety here.

Our interpolant represents the *inverse* of a set of spurious counterexamples. We can use it to refine our abstraction:

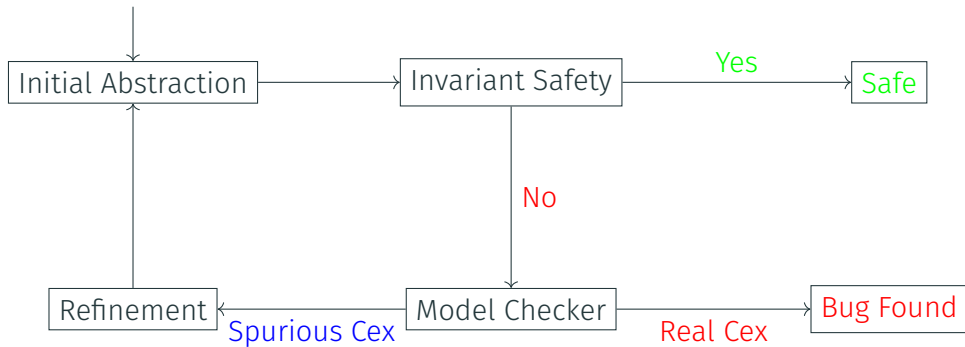
$$a \mapsto a \sqcap I$$

So in the last step, we get the refined abstraction:

$$0 \leq y \leq 1 \wedge 0 \leq z \leq 1 \wedge y + z \leq 1$$

which is sufficient to prove safety here.

In general, we have to add back the interpolant step by step to each point of the trace, then continue the fixpoint computation.



What am I doing with this?

1. Extending interpolation procedures to work with algebraic data types in the presence of recursive functions

What am I doing with this?

1. Extending interpolation procedures to work with algebraic data types in the presence of recursive functions
2. Predicate abstraction for higher-order functional programs

What am I doing with this?

1. Extending interpolation procedures to work with algebraic data types in the presence of recursive functions
2. Predicate abstraction for higher-order functional programs
3. Using invariants generated this way as independently checkable certificates of program correctness

What am I doing with this?

1. Extending interpolation procedures to work with algebraic data types in the presence of recursive functions
2. Predicate abstraction for higher-order functional programs
3. Using invariants generated this way as independently checkable certificates of program correctness

What am I doing with this?

1. Extending interpolation procedures to work with algebraic data types in the presence of recursive functions
2. Predicate abstraction for higher-order functional programs
3. Using invariants generated this way as independently checkable certificates of program correctness

1 and 2 would bring more automation to verifiers like Stainless, while 3 would allow us to increase trust in the system by checking its results using simpler logical systems, like Lisa.

Questions?

Horn Encoding of Programs

- Abstract transition equations are actually just Horn clauses
- A Horn solver implements CEGAR (or something similar) internally
- As long as we can translate our program into an abstract transition system, the verification machinery should just work
- Examples: Eldarica (written in Scala), Spacer (C++; integrated with Z3)

We will look at some examples of Horn encodings if we have time.

Some quick references:

- https://en.wikipedia.org/wiki/Horn_clause
- https://en.wikipedia.org/wiki/Constrained_Horn_clauses

Some resources which have helped me with these topics (in no particular order):

- Samuel R Buss. **“Propositional proof complexity an introduction”**. In: *Computational Logic: Proceedings of the NATO Advanced Study Institute on Computational Logic, held in Marktoberdorf, Germany, July 29–August 10, 1997*. Springer, 1999, pp. 127–178
- Philipp Rümmer, Hossein Hojjat, and Viktor Kuncak. **“Classifying and solving horn clauses for verification”**. In: *Working Conference on Verified Software: Theories, Tools, and Experiments*. Springer. 2013, pp. 1–21
- Nikolaj Bjørner, Arie Gurfinkel, Ken McMillan, and Andrey Rybalchenko. **“Horn clause solvers for program verification”**. In: *Fields of Logic and Computation II: Essays Dedicated to Yuri Gurevich on the Occasion of His 75th Birthday*. Springer, 2015, pp. 24–51
- Kenneth L McMillan. **“Interpolation and model checking”**. In: *Handbook of Model Checking*. Springer, 2018, pp. 421–446